Group No:	19
Group Members (*Student IDs*):	Ameer (20715282) Aarush (20790872) Huda (20801771) Kamila (20675394)

Justification of YOUR Circuitry Diagram Design:

(**NOTE**: No exceeding 2 pages)

- Overview

The main role of the CPU is to perform the fetch, decode, and execute Hack machine instructions. My implementation retains the core, high-level structure of the original nand2tetris CPU .This structure centers on the A and D register, an Arithmetic Logic Unit (ALU), a Program Counter (PC), and the associated control logic.

However, my design modifies the decode and control mechanism to lower the explicit count of top-level Boolean gates, while still maintaining a clear data path. The original CPU design uses around 16 basic logic gates (NOT / AND / OR). My design achieves the same functionality using only 13 gates, which is an approximate 19% reduction. This reduction of gates used improve both time and cost efficiency of the chip, as fewer gates typically mean fewer transistors, lower power consumption, and simpler internal wiring.

- Key Modifications and Comparison with Original CPU design

1. Separated Instruction Type and Simpler Decode:

I explicitly generate *Ainstruction* and *Cinstruction* signals using just one NOT and one AND gate. The load logic for the A register is simplified to *loadAregister* = *Ainstruction* OR *instruction[5]*, replacing the more complicated original decode logic.

2. Reduced gate count in the data and control path:

Original Design	3 NOT, 10 AND, 3 OR (16 Gates)
My Design	2 NOT, 7 AND, 4 OR (13 Gates)
Remarks	This change decrease the hardware's size, complexity, and overall switching activity

3. Simplified A-register data path:

The Mux16 that precedes the A register chooses directly between *ALU out* and *instruction* based on the *Ainstruction* signal. This makes the behavior clearer and reduces the need for intermediate signals.

4. Time-efficient jump logic:

The jump conditions (JGT, JEQ, JLT) are calculated using a minimal gate arrangement that still supports all Hack jump mnemonics. The PC load logic is correct but more concise and cleaner.

- Circuitry Diagram Explanation

The A register output is routed to both *addressM[0..14]* and the select path for the ALU's y-input. The D register's output serves as the x-input for the ALU.The *inM* can also be directed into the ALU via a Mux. The ALU's outputs go to *outM* and also fed back to both A and D registers. The Jump logic uses ALU flags (*zeroflag* and *negativeflag*) to determine whether to load a new value into the PC. The PC increments normally unless a jump is triggered by (*jump=1*) or the system is being reset (*reset=1*).

- Justification, Efficiency, and Reflection

My CPU is completely compatible with the Hack specification while simultaneously improveing hardware efficiency:

- Fewer gates → smaller, cheaper, and lower power usage.
- Fewer logic levels → potentially faster clock speeds.
- Clearer structure of instruction bits → so easier for debugging and readability.
- Modular structure→ easier to maintain or extend.

In conclusion, my CPU design keeps all functional requirements together while showing a more efficient and cleaner hardware implementation.